

SPEIT 科创工作坊讲义

玩转 Linux & Git，从此环境配置不踩坑

引言

很多同学第一次接触开源项目，不是败在"代码看不懂"，而是败在最前面的环境配置：看见 README 很激动，结果第一步就卡在 apt、pip、conda、git clone、代理、权限、版本冲突这些东西上。配了半天，项目没跑起来，心态先崩了。

这份讲义想传递一个核心观念：**环境配置不是目的，跑起项目才是目的。**

Linux、Git、Conda、pip、Homebrew，这些都只是工具。你不需要一开始就把每个工具都学到"专家级"，但需要建立一条足够清晰的主线：先搞清楚你的系统是什么，再搞清楚软件从哪里来，然后搞清楚项目依赖装到哪里，最后搞清楚代码怎么从 GitHub 拉下来、怎么跑、跑不起来怎么排查。这条线打通了，后面学任何项目都会轻松很多。

那我们下面，就先从 WSL 开始吧。

什么是 WSL?

推荐大家直接去看 Microsoft WSL 的官方文档：<https://learn.microsoft.com/zh-cn/windows/wsl/setup/environment>。事实上，无论学什么东西，看官方文档往往是最好的方式（有的时候比问 AI 还高效，毕竟 AI 会有幻觉）

WSL，全称 **Windows Subsystem for Linux**，可以把它理解成：你在 **Windows** 电脑里，开了一个原生 **Linux** 小房间。

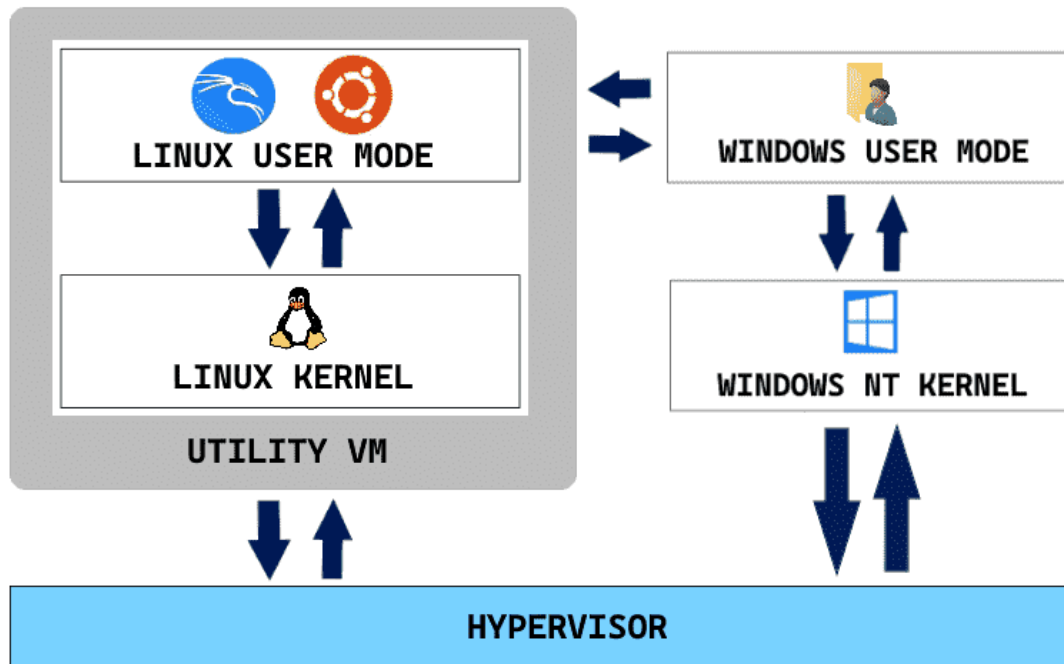
这个比喻很重要。很多初学者会自然地认为："我不是已经在 Windows 上装了 Python / Git / 某个软件了吗，为什么在 WSL 里还找不到？"原因就在这里：**Windows 环境和 WSL 环境是两套不同的用户空间**。虽然共用同一台机器的硬件，但系统文件、包管理方式、命令路径、环境变量很多都不一样。

所以：

- Windows 装的软件，不等于 WSL 里也有；
- WSL 里 `apt install` 的软件，也不等于 Windows 里能直接用；
- Windows 文件管理器里的路径，和 Linux 终端里的路径，写法也常常不一样。

下面这张图简要说明了 Windows 和 WSL 的关系：

WSLv2 ARCHITECTURE



所以 **WSL** 不是"再买了一台电脑", 而是在同一台 **Windows** 机器里额外准备了一套 **Linux** 开发环境。它和 **Windows** 之间既有隔离性（包管理、命令路径、工具链不同），也有协同性（硬件共用、文件可互通、网络可以一起配）。

WSL 和传统虚拟机

很多同学第一次听到 **WSL**, 会问: "这不就是虚拟机吗?" 这个问题很自然, 因为两者确实有相似之处, 但并不一样。

共同点是: 都能让你在 **Windows** 电脑上使用 **Linux** 环境, 都不需要重装系统。但差异挺大。

传统虚拟机 (**VMware**、**VirtualBox**、**Hyper-V**) 通常会模拟出一整台完整的"虚拟电脑"——你要自己分配 **CPU**、内存、磁盘, 自己装完整系统, 然后在一个独立窗口里打开它。更像是"在你的电脑里再开一台电脑"。**WSL** 不是这个思路, 它更像是在 **Windows** 里嵌入一个 **Linux** 用户空间, 重点服务于开发、命令行、工具链、脚本运行这些需求。打开终端就能进, 不需要先启动一个完整的 **Linux** 图形桌面。

资源开销也不一样。虚拟机要跑一个更完整的系统, 内存和磁盘占用更重, 启动维护成本也更高。**WSL** 通常更轻, 启动更快, 和 **Windows** 文件、终端、编辑器之间的衔接也更自然。

另外, 在虚拟机里你经常要处理复制粘贴、共享文件夹、端口映射、代理转发、屏幕分辨率这些细节。**WSL** 的设计目标之一就是把这些"跨系统摩擦"尽量降下来。

为什么推荐 WSL?

上手成本低。不需要先学会创建虚拟机、分配磁盘、装 ISO、调虚拟网卡。装好 WSL，直接进终端就能开始。

和 Windows 日常工作流融合得好。可以继续用 Windows 浏览器查资料、用 VS Code 写代码、用微信，同时把真正的开发和运行环境放在 WSL 里。这对学生和初学者来说非常实用。

更适合跟教程、跑开源项目。绝大多数教学项目、README、安装脚本，默认都更偏 Linux 生态。WSL 能让你在 Windows 机器上，尽量少改动地复用这些材料。

排错路径更清晰。在 Windows 原生里折腾 Python、CUDA、编译链，很多问题会混在一起；放到 WSL 里，很多问题就回到了 Linux 语境，搜资料、照着官方文档做都会更顺。

Windows / WSL / 虚拟机 / 双系统怎么选?

最容易踩的坑，不是"哪一种技术最强"，而是"你当前的目标和它是否匹配"。如果只是想尽快进入 Linux 命令行世界、学 Git、配 Python 环境、把项目跑起来，不同方案的优先级其实差很多。

方案	特点	优点	缺点	适合谁
Windows 原生	完全在 Windows 里开发	门槛最低	很多开源项目偏 Linux，环境差异大，排错容易分散	只做非常轻量的脚本，或暂时不打算进 Linux 生态的人
WSL	在 Windows 里获得接近原生的 Linux 命令行环境	轻量、够用、和 Windows 共存良好	终究不是完整 Linux 桌面；少数底层系统实验场景不如虚拟机	大多数 Windows 初学者、科研同学、做开发和跑项目的人
虚拟机	在 Windows/Mac 里再开一台完整 Linux	隔离性好，系统完整，可快照	更吃资源，安装维护成本更高，文件和网络配置更繁琐	需要完整 Linux 桌面、系统课程、网络实验的人
双系统	直接并列安装 Windows 和 Linux	真正原生性能	安装门槛高，分区和引导容易出问题，切换不方便	已经比较熟悉 Linux，准备长期把 Linux 当主力的人

总结：想最低成本开始学和用，优先 WSL。想做完整系统实验，选虚拟机。想 All in Linux，再考虑双系统。在 Windows 原生里硬扛，通常不是最省心的路。

WSL 容易踩的坑：网络

默认情况下，Windows 和 WSL 的网络并不是完全"无感互通"的。常见现象：

- Windows 上代理已经开了，但 WSL 里下载还是很慢；
- WSL 里启动了一个网页服务，但浏览器访问有问题；
- localhost、端口映射、DNS 解析表现不一致。

如果你用的是比较新的 WSL（WSL2 以上，新安装的基本都是），推荐配置 **mirrored networking**。它让 WSL 和 Windows 尽量共用网络能力，而不是虚拟出一套独立网络。这样很多代理、端口、局域网访问问题都会简单很多。

推荐配置

在 Windows 侧创建文件：

```
C:\Users\<你的用户名>\.wslconfig
```

写入：

```
[wsl2]
networkingMode=mirrored
dnsProxy=true
```

然后重启 WSL：

```
wsl --shutdown
```

更详细的说明可以参考：<https://learn.microsoft.com/zh-cn/windows/wsl/wsl-config>

排查网络问题的思路

如果怀疑 WSL 网络有问题，可以按这个顺序查：

```
# 看看 DNS 是否正常
ping pypi.org

# 看看 GitHub 是否能访问
curl -I https://github.com

# 看看本地代理变量有没有生效
echo $http_proxy
echo $https_proxy
```

如果 curl 很慢、ping 解析失败，优先怀疑网络和代理，而不是先怀疑 Python 或 Git 本身。

关于 Mac

如果你用的是 macOS，那么恭喜你：你的终端世界，本质上和 **Unix / Linux** 非常接近。

这意味着很多 **Linux** 教程里的命令，可以直接学、直接用。对做开发的人来说，Mac 最大的优势不是颜值，而是命令行体验、工具链兼容性和日常使用之间的平衡做得很好。更准确地说，macOS 底层是 **Unix** 系谱，不是 **Linux** 内核本身——但对大多数开发场景来说，这个区别不影响你学 shell、Git、Python 虚拟环境、编译工具链这些基础技能。所以如果你是 Mac 用户，不需要像 Windows 那样先装一层 WSL。你的终端已经足够作为开发入口。

有一个地方不要乱碰：系统 Python

macOS 系统里通常自带 Python 或相关运行时组件。初学者最容易犯的错误之一，就是直接对系统 Python 执行：

```
sudo pip install ...
```

非常不推荐。系统自带的运行时经常服务于操作系统本身或某些系统工具。用 `sudo pip` 去改它，相当于把“系统自己要住的房子”拿来当实验田，轻则依赖混乱，重则系统工具异常。

正确的思路是遵守环境隔离原则，我们在下面会讲到。

包管理器（如 apt、pip）是什么？

包管理器可以理解为系统的“应用商店 + 安装器 + 卸载器 + 更新器”。你当然可以手动去官网找安装包、点下载、点下一步，但那种方式效率低，而且难以复现。命令行包管理器的价值在于：统一安装方式、自动解决依赖、方便脚本化、方便别人复现你的环境。

Ubuntu 的包管理器：apt

在 Ubuntu 里，最常见的系统包管理器是 `apt`。可以把它理解成：**Linux** 系统级软件的官方进货渠道。

最常用的四个命令

```
# 刷新软件包索引
sudo apt update

# 安装软件
sudo apt install git

# 卸载软件
sudo apt remove git

# 搜索软件
apt search python
```

它们分别在做什么？

`sudo apt update` 并不是"更新系统"，而是**更新可安装软件的目录信息**。可以理解成"刷新货架信息"。如果不先刷新，系统有时根本不知道最新的包在哪里。

`sudo apt install git` 才是安装动作。它会去仓库把 **git** 以及它依赖的系统组件一起装好。

`sudo apt remove git` 是卸载——但要注意，卸载软件不一定会清掉所有依赖和配置文件，不要以为 **remove** 就等于"完全回到从前"。

Attention: 有时 apt 会提示你使用 `apt autoremove`。除非你清楚自己在做什么，否则一般不要用这个命令。

`apt search python` 检索仓库中与 Python 相关的软件包。

为什么有些命令前面要加 sudo？

因为系统级安装会修改受保护的目录。`sudo` 的意思是临时以管理员身份执行，也就是俗称的"提权"。输入密码时终端通常不会显示字符，这是正常的，不是键盘坏了。

apt 和 apt-get 有什么区别？

很多老教程喜欢写 `apt-get`，因为它历史更久。现在日常交互式使用推荐 `apt`，它更适合人直接操作，输出也更友好。粗略记住：给人用的时候用 `apt`，老文档和脚本里常见 `apt-get`。两者差异不用抠得很细，初学阶段用 `apt` 就够了。

Mac 的包管理器：Homebrew

macOS 上最常见的包管理器是 **Homebrew**，命令通常写成 `brew`。如果说 Ubuntu 的 `apt` 是系统自带商店，那么 Homebrew 更像是开发者社区给 macOS 补上的那个"应用商店"。

常用命令

```
# 安装软件
brew install git

# 更新 Homebrew 自身及软件索引
brew update

# 升级已安装软件
brew upgrade
```

要安装 Git、wget、ffmpeg、cmake、node、python、tmux 这类开发工具，通常优先考虑 Homebrew。它会把软件安装到一套比较规范、可管理的位置，而不是让你到处手工下载。

一个常见的坏习惯

很多 Mac 新手会混着用三套来源装软件：官网 dmg、Homebrew、系统自带。这不是不能用，而是容易失控。更稳的策略是：命令行开发工具尽量统一用 **Homebrew** 管理。

番外：Ubuntu 24.04 为什么不让你直接 `pip install`？

你在新版本 Ubuntu 里可能会看到这样的报错：

```
pip install numpy
```

然后出现：

```
error: externally-managed-environment
```

这不是你的电脑坏了。系统在明确告诉你：不要直接往系统 **Python** 里装包。

背后的原因：PEP 668

PEP 668 的核心思想很简单：如果这个 **Python** 基础环境是由操作系统包管理器维护的，那 `pip` 默认就不应该随便改它。原因有两个：一是保护系统稳定性——Ubuntu 系统里一些工具本身依赖 **Python**，如果用 `pip` 把底层库版本改掉，可能影响系统工具运行；二是避免项目间冲突——不同项目需要的包版本可能完全不同，都往同一个全局环境装，最后一定会乱。

正确的做法

不要和这个限制对抗，顺着它走。现代 **Python** 开发最重要的一条原则就是：

每个项目使用自己的独立环境。

常见做法有三种：**Conda**（适合科研、数据科学、深度学习、多语言依赖场景）、**venv**（**Python** 标准库自带，轻量直接）、**pipx**（适合安装命令行工具，而不是项目依赖）。

Conda 和 pip 到底怎么分工？

这是环境配置里最容易搞混、也最值得彻底理解的部分。一个实用的经验法则：

*Conda 负责建环境，pip 负责装 **Python** 包。*

也就是说：**Conda** 是盖房子的，**pip** 是买家具的。**Conda** 搭建出隔离环境，决定 **Python** 版本以及一些底层依赖怎么放；**pip** 往已经建好的环境里安装 **Python** 包。

为什么不能乱混？

因为 Conda 不只管理 Python 包，它还管理更底层的二进制依赖。如果先用 `pip` 装了一堆东西，再用 Conda 改底层依赖，Conda 可能会为了满足约束而重排整个环境，反过来也一样。

更根本的原因是，`conda install` 和 `pip install` 走的是**两套不同的依赖体系**。`conda install` 解决的是 Conda 自己维护的整套包生态，安装的不只是 Python 包，还可能包括 `libstdc++`、CUDA 相关库、编译好的二进制 Runtime 文件，它站在"整个环境"的角度做依赖求解。`pip install` 解决的是 PyPI 上 Python 包之间的依赖关系，主要关心的是"这个 Python 包依赖哪些别的 Python 包"，不负责系统级库和底层运行时的一体化管理。

你可以这样理解：**Conda 管的是整栋楼的结构、水电、门窗；pip 管的是这个房间里再放哪些桌椅和电器**。如果先让 `pip` 往房间里塞了很多东西，再让 Conda 回头改整栋楼的结构，Conda 未必能完全理解 `pip` 刚才做了什么；反过来也不例外。这就是为什么"看起来都装成功了"，环境最后却可能悄悄变脏、变脆弱，甚至到运行时才报错。

更稳妥的习惯是：先用 Conda 建好环境，再用 `pip` 安装项目要求的 Python 依赖；装完之后尽量不要频繁来回用 Conda / `pip` 改大件。

Workflow

```
# 1. 创建环境
conda create -n my-project python=3.11 -y

# 2. 激活环境
conda activate my-project

# 3. 安装项目依赖
pip install -r requirements.txt
```

常用 Conda 命令

```
# 查看已有环境
conda env list

# 创建新环境
conda create -n my-env python=3.11 -y

# 激活环境
conda activate my-env

# 退出当前环境
conda deactivate

# 删除环境
conda remove -n my-env --all -y
```


为什么换镜像源有用？

很多包管理器默认使用国外源站。这本身没有问题，但物理距离、网络路径、访问限制等因素会让下载体验很差。换镜像源的思路是：本来你要跨洋去仓库拿货，现在换成国内同步过来的分发点。

有一点要注意：**镜像源主要解决"下载慢"，不解决"根本无法访问 GitHub"这类问题。** 这两件事不要混为一谈。

pip 换源

推荐命令

```
pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple
pip config list
```

第一行把 pip 的默认包索引改成清华镜像，第二行检查当前 pip 配置是否生效。

临时换源

如果不想改全局配置，可以在某次安装时临时指定：

```
pip install -i https://pypi.tuna.tsinghua.edu.cn/simple -r requirements.txt
```

适合公共机器、临时环境，或者你不想永久修改配置的时候。

Conda 换源

基本配置

```
conda config --add channels
https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/main/
conda config --add channels
https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/free/
conda config --set show_channel_urls yes
```

查看当前配置

```
conda config --show channels
```

有一点要说清楚

Conda 生态比 pip 更复杂，涉及多个 channel。不同学校、实验室、项目会有不同习惯：有人用默认 channel，有人用 conda-forge，有人混用。换源只是加速手段，不是环境管理的全部。你真正需要知道的，是这个环境里的包究竟从哪里来。

Ubuntu 系统源换源

Ubuntu 24.04 适配写法

```
sudo sed -i 's|http://archive.ubuntu.com|https://mirrors.tuna.tsinghua.edu.cn|g' /etc/apt/sources.list.d/ubuntu.sources
sudo sed -i 's|http://security.ubuntu.com|https://mirrors.tuna.tsinghua.edu.cn|g' /etc/apt/sources.list.d/ubuntu.sources
sudo apt update
```

Ubuntu 22.04 及更早版本

```
sudo sed -i 's|http://archive.ubuntu.com|https://mirrors.tuna.tsinghua.edu.cn|g' /etc/apt/sources.list
sudo sed -i 's|http://security.ubuntu.com|https://mirrors.tuna.tsinghua.edu.cn|g' /etc/apt/sources.list
sudo apt update
```

为什么 24.04 和旧版本写法不一样？

新版本 Ubuntu 的源配置文件组织方式有所变化。旧版本很多教程直接改 `/etc/apt/sources.list`，而新版本有时默认使用 `/etc/apt/sources.list.d/ubuntu.sources`。从网上抄命令时，一定要先确认自己的系统版本，不要机械照搬。

换源后如何验证？

```
apt policy
# 或者直接
sudo apt update
```

如果输出中的下载地址变成了你设置的镜像站，说明基本生效了。

Homebrew 换源

PPT 中给出的方式是修改 Homebrew 仓库地址：

```
git -C "$(brew --repo)" remote set-url origin
https://mirrors.tuna.tsinghua.edu.cn/git/homebrew/brew.git
```

这类做法本质上是在调整 Homebrew 自身 Git 仓库的上游地址。但要提醒大家：**Homebrew 的生态和官方策略会变化，镜像可用性也会波动**。实际使用时，优先参考当前镜像站说明与 Homebrew 官方文档，不要把某条旧命令当成永远有效的标准答案。如果你刚入门，当前网络也正常，可以先不折腾 brew 换源。先把项目跑起来，比把每个工具都调到最优更重要。

Git 是什么？

Git 不是程序员专属黑魔法，本质上是一套**版本管理系统**。它能帮你保存代码历史、回到过去、和别人协作，让你不再害怕"我把文件改坏了怎么办"。

没有 Git 的时候，很多同学的版本管理方式其实是：

- final.py
- final_really_final.py
- final_really_final_v2.py
- final_really_final_v2_new.py

这当然也算"版本管理"，但管得非常痛苦。Git 的价值在于把这些历史变成一条清晰、可查询、可回退的记录。

Git 的四层工作流：一定要形成画面感

初学 Git，最重要的不是背命令，而是建立这四层的心智模型：

1. 工作区（**Working Directory**）：你正在改的文件；
2. 暂存区（**Staging Area**）：你准备放进下一次提交的修改；
3. 本地仓库（**Local Repository**）：已经提交、已经存档的历史；
4. 远程仓库（**Remote Repository**）：比如 GitHub 上的云端版本。

对应的关键命令

```
# 把修改加入暂存区
git add .

# 把暂存区内容提交到本地历史
git commit -m "write a clear message"

# 推送到远程 GitHub
git push
```

git add 不是"提交"，而是把你选中的改动放进"待打包区"。git commit 才是正式拍快照，把当前版本写进本地历史。git push 则是把本地历史同步到远程仓库。理解这三者的区别，能帮你避免很多低级混乱。

常用 Git 命令

```
# 初始化仓库
git init

# 查看当前状态
git status

# 添加单个文件
git add filename.py

# 添加所有修改
git add .

# 提交
git commit -m "your message"

# 查看远程仓库
git remote -v

# 拉取最新代码
git pull

# 推送代码
git push
```

为什么 `git status` 特别重要？

因为它几乎就是你的 Git 仪表盘。当你不知道自己在哪个分支、有哪些文件改了、哪些已经暂存、接下来该做什么的时候，第一反应应该是：

```
git status
```

养成这个习惯，很多 Git 恐惧症会立刻减轻。

连接 GitHub

连接方式分两类：

- **HTTPS**：每次 push 时都需要手动输入一遍 token；
- **SSH**：配置一次密钥后，后续使用更顺手。

长期在自己电脑上开发的话，SSH 往往更省心。

SSH 的直觉理解

你可以把 SSH key 理解成一对钥匙：私钥留在你自己的电脑上，公钥交给 GitHub。以后你访问 GitHub 时，GitHub 会验证"你是不是持有那把正确的私钥"。只要验证通过，就允许你进行 Git 操作，不需要频繁手动输密码。

生成 SSH Key

```
ssh-keygen -t ed25519 -C "你的邮箱"
```

一路回车通常就可以。如果系统太老、不支持 ed25519，再考虑退回 RSA (-t id_rsa)；但现在一般优先推荐 ed25519。

查看公钥并复制

```
cat ~/.ssh/id_ed25519.pub
```

把输出中那一整行复制下来。

添加到 GitHub

进入 GitHub: Settings → SSH and GPG keys → New SSH Key，把刚才复制的公钥贴进去即可。

测试是否成功

```
ssh -T git@github.com
```

如果看到欢迎信息，说明基本配置成功。

Git config

```
git config --global user.name "你的名字"  
git config --global user.email "你的邮箱"
```

每次提交都需要记录"是谁提交的"。如果不提前配置，后面提交时经常会收到提醒，或者提交信息看起来不规范。

查看当前配置：

```
git config --global --list
```

GitHub 很慢怎么办？

这要分情况。

如果是"下载很慢"

可能是网络路径、代理、DNS 问题。可以考虑：正确配置代理、WSL mirrored networking、换下载源（针对 pip / conda / apt，不是 GitHub 本体）。

如果是"Git 操作慢"

比较稳的思路是直接让 Git 走本机代理：

```
git config --global http.proxy http://127.0.0.1:7890
git config --global https.proxy http://127.0.0.1:7890
```

取消代理：

```
git config --global --unset http.proxy
git config --global --unset https.proxy
```

如果在 Linux 里开发

还可以在 shell 配置里加环境变量，写进 ~/.bashrc 或 ~/.zshrc：

```
export http_proxy="http://localhost:7890"
export https_proxy="http://localhost:7890"
```

写完后重新加载：

```
source ~/.bashrc
```

或者重新开一个终端。

为什么不推荐折腾 hosts、野生镜像站之类的方法？

因为那类方案不稳定、过期快、容易引入新的解析问题，有时还涉及安全风险。

实战项目：跑起一个 Streamlit Demo

参考下发文件中的 04-项目复现流程.md。

遇到报错时，不要只盯着最后一行

很多初学者一看到红字就慌，然后只把最底下一句复制给别人。但实际上，报错信息通常是有结构的：前面是上下文（程序在做什么），中间是调用链（问题从哪里传下来），最后才是核心异常类型和提示。排错时应该养成这个习惯：先看你执行的是哪条命令；再看报错属于哪一类（网络、权限、路径、版本、依赖、端口）；最后再决定怎么搜、怎么问人。主要的分类如下：

网络问题 — 常见关键词：`timeout`、`connection refused`、`proxy`、`SSL`、`name resolution`。优先检查网络、代理、DNS。

权限问题 — 常见关键词：`permission denied`、`operation not permitted`。优先检查 `sudo`、文件权限、安装位置。

路径问题 — 常见关键词：`no such file or directory`、`command not found`。优先检查当前目录、文件名、`PATH`。

Python 依赖问题 — 常见关键词：`ModuleNotFoundError`、`ImportError`、`version conflict`、`externally-managed-environment`。优先检查虚拟环境、`pip` 安装位置、`requirements`。

排错思路：最小化问题

环境问题最怕"同时动十个东西"。你一会儿换 Python、一会儿改 `PATH`、一会儿改代理、一会儿手动删目录，最后自己都不知道哪一步起了作用，哪一步制造了新问题。

更稳的方法：每次只改一个关键变量；改完就验证；验证失败就回退；尽量保留一个"最小可复现"的错误现场。

比如你怀疑 `pip` 源有问题，不要先把 `apt` 源、`conda` 源、`shell` 配置、`SSH` 配置一锅端。先单独验证：

```
pip install requests
```

如果这一步都不通，再继续查 `pip` 层；如果通，那说明问题可能在项目依赖本身，而不是 `pip` 基础配置。

最后的办法：Revert!

很多同学总觉得"我已经配了半小时了，现在删环境重来是不是前功尽弃"。其实恰恰相反：在错误环境上继续叠补丁，往往才是真正的时间黑洞。

如果你已经把某个 `Conda` 环境折腾到自己都不清楚里面装了什么、删了什么、换了什么，那最省时间的做法通常就是：

```
conda deactivate
conda remove -n my-env --all -y
conda create -n my-env python=3.11 -y
conda activate my-env
```

把原来的环境删了，建一个新的。环境是可重建的，时间才是最贵的。

常见问题速查

`command not found`

说明当前终端找不到这个命令。优先检查：软件是否真的安装了、当前环境是否激活、PATH 是否正确。

`Permission denied`

说明权限不够，或者文件不可执行。先不要盲目 `sudo`，想清楚你到底是在做系统级操作，还是只是路径不对。

`externally-managed-environment`

说明你在系统 Python 里直接装包，被 PEP 668 拦住了。正确做法是改用 Conda 或 venv。

`ModuleNotFoundError`

说明当前 Python 环境里没有这个包。优先检查是不是装到了错误的 Python 环境中，或者用 `pip install` 安装缺失的包。

`git push` 失败

区分一下：是认证失败（检查 SSH / Token），还是网络失败（检查代理），还是分支冲突（先 `git pull` 看看）。

一些原则

系统环境和项目环境分开。系统层工具交给 `apt / brew`，项目层依赖交给 `Conda / venv + pip`。不要把所有东西都往全局环境里塞。

Git 先理解工作流，再背命令。 `add`、`commit`、`push` 分别在不同层。先有工作区 → 暂存区 → 本地仓库 → 远程仓库的画面感，再学高级用法。

排错靠缩小问题，不靠玄学。 看报错、看环境、看路径、看网络。一次只改一个变量。必要时删环境重建。

References：推荐进一步学习的网站与文档

下面这些资源尽量选了官方文档、经典教程或质量较高的学习网站。虽然按照现在大型 LLM 的能力（GPT、Claude、Gemini 等），基本上都能给你一个不错的回答，但读文档的能力，依然是现代开发中不可或缺的一一让 LLM 全面接管你的东西，目前还是有很大的安全风险的。

Git 学习

1. **Learn Git Branching** — 非常适合初学者的交互式学习网站，特别适理解分支、合并、rebase 等概念。
<https://learngitbranching.js.org/>
2. **Pro Git** (官方推荐在线书) — 系统、完整、免费，适合从基础一路学到较进阶内容。
<https://git-scm.com/book/en/v2>
3. **Git Reference** — 忘了某个命令怎么用时，当查询手册用。
<https://git-scm.com/docs>
4. **GitHub SSH 文档** — 配 SSH key 时非常有用，建议直接看官方步骤。
<https://docs.github.com/en/authentication/connecting-to-github-with-ssh>

Linux / Shell 学习

5. **The Missing Semester of Your CS Education** — MIT 的经典课程，覆盖 shell、Git、编辑器、自动化等开发者工具链。
<https://missing.csail.mit.edu/>
6. **Missing Semester: Shell** — 想先集中补 shell，这一讲非常值得从头看。
<https://missing.csail.mit.edu/2020/course-shell/>
7. **Linux Journey** — 偏入门风格，适合系统补 Linux 基础知识。
<https://linuxjourney.org/>

WSL / Windows 开发环境

8. **Microsoft WSL 文档** — WSL 的总入口，很多配置问题都可以从这里查。
<https://learn.microsoft.com/en-us/windows/wsl/>
9. **WSL Networking 文档** — mirrored networking、DNS、代理相关问题可以重点看这一页。
<https://learn.microsoft.com/en-us/windows/wsl/networking>
10. **WSL 配置文档 (.wslconfig / wsl.conf)** — 查配置项怎么写。
<https://learn.microsoft.com/en-us/windows/wsl/wsl-config>

Python / pip / 虚拟环境

11. **pip Configuration** 官方文档 — 想搞清楚 pip config、index-url 等配置怎么生效，看这份最稳。
<https://pip.pypa.io/en/stable/topics/configuration/>
12. **pip config 命令文档** — 查具体子命令写法。
https://pip.pypa.io/en/stable/cli/pip_config/
13. **Python 官方 venv 文档** — 推荐至少读一遍。
<https://docs.python.org/3/library/venv.html>
14. **PyPA: 用 pip 和 venv 安装包指南** — 偏实践导向，适合想走标准 Python 路线的同学。
<https://packaging.python.org/guides/installing-using-pip-and-virtual-environments/>

15. **PEP 668** — 想从根源搞明白为什么 Ubuntu 24.04 不让你全局 pip，直接看规范。

<https://peps.python.org/pep-0668/>

Conda / 环境管理

16. **Conda 官方文档**

<https://docs.conda.io/>

17. **Conda: Managing Environments** — 创建、激活、删除、导出环境时最常查的一页。

<https://docs.conda.io/projects/conda/en/stable/user-guide/tasks/manage-environments.html>

macOS / Homebrew

18. **Homebrew 官方主页**

<https://brew.sh/>

19. **Homebrew Installation 文档**

<https://docs.brew.sh/Installation>

20. **Homebrew Documentation** — 查 brew 命令、FAQ、常见问题时好用。

<https://docs.brew.sh/>

说在最后

真正把你和别人拉开差距的，不是你会不会背十几条命令，而是你是否形成了这套能力：看到新项目时，知道先建环境再装依赖；看到 Git 仓库时，知道怎么 clone、commit、push；看到报错时，知道先区分网络、权限、路径还是依赖；看到环境乱了时，知道什么时候该修、什么时候该删掉重建。

这套能力一旦形成，以后再学任何框架、任何语言、任何开源项目，都会轻松很多。环境配置不再是绊脚石，而会变成你进入真实工程实践的第一道门。祝大家都能真正做到：不只是"把命令跑过一遍"，而是真的能从零开始跑起自己的第一个真实项目。